

4-Week High-Impact DSA Plan

\

WEEK 1 – Foundations + Arrays, Strings, Hashing

Objective: Build strong fundamentals and confidence

Core Concepts to Revise

- Time Complexity:
 $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$
- Space Complexity
- Recursion basics
- Java Collections overview:
 - ArrayList
 - HashMap
 - HashSet

Data Structures

1. Arrays
2. Strings
3. Hashing (Map / Set)

Must-Know Algorithms & Techniques

- Two Pointer Technique
- Sliding Window
- Frequency counting (HashMap)
- Prefix Sum

Most Important Interview Questions

Practice **15–20 questions total**, not more.

Arrays

- Two Sum
- Best Time to Buy and Sell Stock
- Maximum Subarray (Kadane's Algorithm)
- Move Zeroes
- Merge Sorted Array
- Majority Element

Strings

- Valid Anagram
- Longest Substring Without Repeating Characters
- Palindrome Check
- String Compression
- Reverse Words in a String

Hashing

- Subarray Sum Equals K
- Contains Duplicate
- First Non-Repeating Character

Deliverable by End of Week 1

- You should **instantly recognize** when to use:
 - HashMap vs Array
 - Two pointers vs brute force
 - Able to explain:
 - Why a solution is $O(n)$ instead of $O(n^2)$
-

WEEK 2 – Linked List, Stack, Queue, Recursion

Objective: Handle pointer-based structures confidently

Data Structures

1. Singly Linked List
2. Stack
3. Queue
4. Recursion

Must-Know Algorithms & Patterns

- Fast & Slow Pointer (Floyd's cycle detection)
- Stack-based problems
- Recursive thinking

Most Important Interview Questions

Linked List

- Reverse a Linked List
- Detect Cycle in Linked List
- Merge Two Sorted Lists
- Remove Nth Node from End
- Middle of Linked List

Stack

- Valid Parentheses
- Next Greater Element
- Min Stack
- Implement Stack using Array

Queue

- Implement Queue using Stack
- Circular Queue (conceptual)

Recursion

- Fibonacci (with optimization)
- Factorial
- Reverse String using recursion

Interview Tip

Interviewers often check **how you handle edge cases**:

- Empty list
- Single node
- Null pointer

Be vocal while solving.

Deliverable by End of Week 2

- No fear of pointers
- Clear explanation of recursion stack
- Ability to convert recursive logic to iterative (bonus)

WEEK 3 – Trees, Binary Search, Sorting

Objective: Crack mid-level interview questions

Data Structures

1. Binary Tree
2. Binary Search Tree (BST)

Must-Know Algorithms

- Binary Search (Iterative & Recursive)
- Tree Traversals:
 - Inorder
 - Preorder
 - Postorder
 - Level Order (BFS)
- Sorting:
 - Merge Sort (important)
 - Quick Sort (conceptual)

Most Important Interview Questions

Binary Search

- Binary Search on Sorted Array
- First & Last Occurrence
- Search in Rotated Sorted Array
- Square Root of a Number

Trees

- Maximum Depth of Binary Tree
- Invert Binary Tree
- Validate BST
- Lowest Common Ancestor
- Level Order Traversal

Sorting

- Explain Merge Sort
- When to use Quick Sort vs Merge Sort

Deliverable by End of Week 3

- You can draw tree problems on paper
- Comfortable explaining recursion in trees
- Strong binary search intuition

WEEK 4 – Graphs, Dynamic Programming, Revision + Mock Interviews

Objective: Be interview-ready

Graphs (Basic Only – Enough for Freshers)

- Representation (Adjacency List)
- BFS
- DFS
- Detect cycle (conceptual)

Important Questions

- Number of Islands
- BFS vs DFS difference
- Connected Components

Dynamic Programming (VERY IMPORTANT)

Do NOT skip DP, but keep it limited.

Concepts

- Overlapping subproblems
- Memoization vs Tabulation

Must-Do Problems

- Fibonacci (DP)
- Climbing Stairs
- Coin Change (basic)
- 0/1 Knapsack (conceptual)

Final Week Activities

- Revise **top 50 problems only**
- Do **mock interviews**
- Practice explaining:
 - Approach
 - Complexity
 - Edge cases

What Interviewers WILL Ask You

- “Why did you choose this approach?”
- “Can you optimize it?”
- “What is the time complexity?”

- “What happens if input is very large?”

Recommended Practice Platforms

- LeetCode (Primary)
- GeeksforGeeks (Concept clarity)
- HackerRank (Warm-up)

How This Aligns with Spring AI / ML Roles

- DSA tests your **thinking ability**
- Spring + AI interviews check:
 - API design
 - Backend logic
 - Optimization thinking

Strong DSA = strong backend engineer.